

Lab3 实验报告

实验说明

实验目的

- 熟练使用 Verilog 完成数字控制系统设计与调试。
- 采用有限状态机（FSM）实现复杂控制逻辑，提升代码可维护性。
- 实现按键去抖、上升沿检测、7 段数码管多路复用显示与 LED 指示等常用模块。
- 在 Vivado 环境中验证设计功能，并为 FPGA（Nexys4 DDR）生成可下载的 bitstream。

实验内容

- 支持 6 种商品，价格（元）：0.5、1.0、1.5、2.0、6.5、13.0。
- SW[0:2] 选择商品，4 个按钮输入货币（5 元/2 元/1 元/5 角）、一个**清零键（补充亮点）**。
- 用数码管显示商品编号与已付金额；6 个 LED 对应 6 个商品购买成功标志。
- 实现按键去抖与系统复位，生成可下载 bitstream。

实验分析

本次实验从模块化设计和可验证性出发，优先将功能划分为去抖、边沿检测、状态机与显示四个子模块。实验思路是先用小范围的单元仿真验证去抖与边沿检测的鲁棒性，再把它们作为输入驱动 FSM，最后完成显示与交互逻辑。调试过程中发现仿真与上板行为的主要差异在于按键脉冲宽度与数码管极性，针对性地在 testbench 中引入 SIM_ACCEL / C_press、在 RTL 中确保 rstn 的完整性，从而提高了回归测试的稳定性。整体调试遵循“复现 → 定位 → 假设 → 修复 → 验证”的闭环，优先解决会导致功能错误的时序与复位问题，然后再优化显示与用户体验逻辑，确保设计既可仿真快速回归又能在板上稳定运行。

设计思路

1. 单位选择

以角作为内部计算单位，避免浮点运算：所有价格和金额均用整数表示；

2. 输入映射

商品选择 (Switch)

- SW[0:2]: 3 位二进制编码选择商品 1-6
 - 001 → 商品 1 (0.5 元 = 5 角)
 - 010 → 商品 2 (1.0 元 = 10 角)
 - 011 → 商品 3 (1.5 元 = 15 角)
 - 100 → 商品 4 (2.0 元 = 20 角)
 - 101 → 商品 5 (6.5 元 = 65 角)
 - 110 → 商品 6 (13.0 元 = 130 角)
 - 其它值视为未选择商品

货币输入 (Button)

- BTNL: 投入 5 元 (50 角)
- BTNR: 投入 2 元 (20 角)
- BTNU: 投入 1 元 (10 角)
- BTND: 投入 5 角 (5 角)
- BTNC: 清零/复位

3. 输出显示

数码管

- D0: 已付金额角位
- D1: 已付金额元个位 (右侧小数点, 形成"元.角"格式)
- D2: 已付金额元十位
- D3: 已付金额元百位
- D4: 商品编号个位
- D5: 商品编号十位

LED 指示

- LED[0..5] 分别对应商品 1..6 的购买成功指示（购买成功后先闪烁若干次再常亮）。

4. 防抖处理

本设计在按键输入端采用**硬件去抖（debounce）模块**以保证 FSM 只对稳定的按键事件响应。去抖设计由三部分组成：**同步寄存器、稳定计数器、上升沿检测**。关键点如下：

原理概述

- 机械按键在按下/释放瞬间会发生高频抖动，直接采样会产生多次误触。
- 采用时序稳定判定：先用双寄存器同步输入到时钟域，消除亚稳态；然后只有当同步后的输入在连续 N 个时钟周期内保持相同值，才认为该按键已稳定，从而更新去抖输出。最后用上升沿检测产生单周期脉冲供 FSM 使用。

模块构成

- 同步： $\text{sync0} \leq \text{in}$; $\text{sync1} \leq \text{sync0}$; 在时钟上升沿执行。
- 计数器：当 $\text{sync1} \neq \text{out}$ 时开始计数，计数达到 N 则 $\text{out} \leq \text{sync1}$ 并清零计数；否则计数清零。
- 上升沿检测：对去抖输出使用 prev 寄存器生成 $\text{rise} = \text{out} \ \& \ \sim\text{prev}$ ，确保一次按下只产生一个脉冲事件。

5. 关键参数与时序

- DEBOUNCE_CYCLES（源码参数）：默认为 1_000_000，约等于 10 ms（在 100 MHz 时钟下）。该参数决定按键稳定判定窗口。
- BLINK_INTERVAL（源码参数）：默认为 50_000_000，约等于 0.5 s（在 100 MHz 时钟下）。用于 LED 或数码管闪烁周期。
- 在仿真中引入了仿真加速因子 SIM_ACCEL（仅 testbench 使用），会对上述时间进行缩短以加速回归，但按键脉冲需保证大于去抖窗口（testbench 中通过 MIN_PRESS_CYCLES 强制）。

6. 工作流程（FSM 实现）

1. IDLE 状态：系统初始态或未选择商品

- 条件：SW[2:0] 选择有效商品（001-110）

- 动作：进入 WAIT_COIN 状态，显示商品编号

2. WAIT_COIN 状态：等待投币并累加

- 输入：按键脉冲（5 元/2 元/1 元/5 角）
- 动作：累加 balance，实时显示在数码管
- 条件：balance >= current_price
- 转移：进入 PURCHASE 状态

3. PURCHASE 状态：触发购买（一次性操作）

- 动作：
 - 设置 success_flags[product_sel] = 1
 - 启动 LED 闪烁 → 亮点：为增强视觉效果，小灯先闪烁3次再稳定亮起（blink_cnt = 3，周期 0.5s）
 - 计算找零金额：change_amount = balance - current_price
- 条件：若 change_amount > 0
- 转移：进入 CHANGE 状态 或 回到 WAIT_COIN

4. CHANGE 状态：找零显示

→ 增加亮点：若用户的货币总额大于商品价格，右侧数码管会先闪烁显示需要找零的金额数，之后回归显示原先的货币总数；若用户货币总额恰好等于商品价格，后续再按按钮，则不会触发找零逻辑。

- 动作：数码管闪烁显示找零金额
- 计时：5 秒倒计时
- 转移：倒计时结束后恢复显示货币总和

5. 清零处理（优先级最高）(reset)

- 任何状态下按 BTNC 立即清零 balance 和 success_flags
- FSM 返回 IDLE 状态
- 处理路径（信号流）：
 - 物理按键 BTNC → debounce 模块（同步 + 稳定计数）→ 去抖输出 btn_clear_db（稳态电平）→ edge_detect（上升沿检测）→ btn_clear_pulse（单周期同步脉冲）

- 其中 `debounce` 与 `edge_detect` 均在时钟域内工作，`edge_detect` 产生的 `rise` 脉冲为同步单周期信号，可安全在主时序逻辑中直接使用。
- 时序与优先级：
 - `btn_clear_pulse` 是同步单周期脉冲（由 `edge_detect` 在时钟上产生）；主控制逻辑在 `posedge CLK100MHZ` 的时序块中检测该脉冲并执行清零操作，因此清零在检测到脉冲后的时钟沿生效（同步行为）。
 - 在源码的主时序 `always` 块中，清零分支（`if (btn_clear_pulse)`）位于逻辑处理的最前面——即清零具有最高业务优先级：当清零脉冲出现时，清零操作会在该时钟周期优先执行，从而抑制同一周期内的投币/购买更新，避免竞态或不一致状态。

关键特性

- **FSM 控制流程**：状态转移逻辑清晰，易于验证和扩展
- **超额支付允许**：`balance >= current_price` 即可购买（不要求精确匹配）
- **可重复投币**：按两次 2 元按钮 = 4 元
- **即时反馈**：金额达标立即点亮 LED 并触发闪烁
- **LED 闪烁提示**：购买成功时 LED 闪烁 3 次（0.5s 周期）
- **找零显示**：超额投币时数码管闪烁显示找零金额 **3 次**（每次 0.5 秒，共 1.5 秒），闪烁完成后恢复显示货币总值

Vivado 仿真

撰写激励文件，在 Vivado 上进行仿真，可正确实现功能：

```

=== TEST1: exact payment product1(0.5Y) pay 0.5Y ===
[5000] state=0 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111110
15000: TB - select product SW=001 (id=1)
[25000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[155000] state=1 | balance=5 (0.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[165000] state=2 | balance=5 (0.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[175000] state=1 | balance=5 (0.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[685000] state=1 | balance=5 (0.5 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=111111101
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_vending_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 1138.629 ; gain = 0.000
run all
[1195000] state=1 | balance=5 (0.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[1695000] state=1 | balance=5 (0.5 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=111111101
[2205000] state=1 | balance=5 (0.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[2705000] state=1 | balance=5 (0.5 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=111111101
10015000: TB - press 0.5Y button (cycles=1000)
10035000: TB - CHECK: TEST1 exact pay
[PASS] LED state: 000001
[PASS] balance: 5 (0.5 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

购买商品1, 付0.5元: 正好购买

```

=== TEST2: overpay product1(0.5Y) pay 1Y (expect change 0.5Y) ===
[10175000] state=1 | balance=0 (0.0 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=111111101
[10185000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
20035000: TB - press CLEAR button (cycles=1000)
20055000: TB - select product SW=001 (id=1)
[20195000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[20205000] state=2 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=111111101
[20215000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=1 | change_amount=5 | AN=111111101
[20725000] state=1 | balance=10 (1.0 Y) | LED=000001 | change_display=1 | change_amount=5 | AN=111111111
[21235000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=1 | change_amount=5 | AN=111111101
[21735000] state=1 | balance=10 (1.0 Y) | LED=000001 | change_display=1 | change_amount=5 | AN=111111111
[22245000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=1 | change_amount=5 | AN=111111101
[22745000] state=1 | balance=10 (1.0 Y) | LED=000001 | change_display=1 | change_amount=5 | AN=111111111
[23215000] state=1 | balance=10 (1.0 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=111111101
30055000: TB - press 1Y button (cycles=1000)
INFO: post-pay state (no assert on change_display): change_display=0 change_amount=0 change_blink_cnt=0
30095000: TB - CHECK: TEST2 overpay result
[PASS] LED state: 000001
[PASS] balance: 10 (1.0 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

购买商品1, 付1元: 超额依然可以购买

```

=== TEST3: big overpay product4(2.0Y) pay 5Y ===
[30235000] state=1 | balance=0 (0.0 Y) | LED=000001 | change_display=0 | change_amount=0 | AN=11111101
[30245000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
40095000: TB - press CLEAR button (cycles=1000)
40115000: TB - select product SW=100 (id=4)
[40255000] state=1 | balance=50 (5.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[40265000] state=2 | balance=50 (5.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[40275000] state=1 | balance=50 (5.0 Y) | LED=000000 | change_display=1 | change_amount=30 | AN=11111101
[40785000] state=1 | balance=50 (5.0 Y) | LED=001000 | change_display=1 | change_amount=30 | AN=11111111
[41295000] state=1 | balance=50 (5.0 Y) | LED=000000 | change_display=1 | change_amount=30 | AN=11111101
[41795000] state=1 | balance=50 (5.0 Y) | LED=001000 | change_display=1 | change_amount=30 | AN=11111111
[42305000] state=1 | balance=50 (5.0 Y) | LED=000000 | change_display=1 | change_amount=30 | AN=11111101
[42805000] state=1 | balance=50 (5.0 Y) | LED=001000 | change_display=1 | change_amount=30 | AN=11111111
[43275000] state=1 | balance=50 (5.0 Y) | LED=001000 | change_display=0 | change_amount=0 | AN=11111101
50115000: TB - press 5Y button (cycles=1000)
INFO: post-pay state (no assert on change_display): change_display=0 change_amount=0 change_blink_cnt=0
50165000: TB - CHECK:          TEST3 big overpay result
[PASS] LED state: 001000
[PASS] balance: 50 (5.0 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

购买商品4, 付5元: 超额依然可以购买

```

=== TEST4: debounce quick double-press ===
[50305000] state=1 | balance=0 (0.0 Y) | LED=001000 | change_display=0 | change_amount=0 | AN=11111101
[50315000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
60165000: TB - press CLEAR button (cycles=1000)
60185000: TB - select product SW=010 (id=2)
[60325000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[60335000] state=2 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[60345000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[60855000] state=1 | balance=10 (1.0 Y) | LED=000010 | change_display=0 | change_amount=0 | AN=11111101
[61365000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[61865000] state=1 | balance=10 (1.0 Y) | LED=000010 | change_display=0 | change_amount=0 | AN=11111101
[62375000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[62875000] state=1 | balance=10 (1.0 Y) | LED=000010 | change_display=0 | change_amount=0 | AN=11111011
INFO: balance now=10 (angles)
80215000: TB - CHECK:          TEST4 debounce check
[PASS] LED state: 000010
[PASS] balance: 10 (1.0 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

购买商品2, 付1元: 正好购买

```

=== TEST5: insufficient pay product6 (no LED) ===
[80355000] state=1 | balance=0 (0.0 Y) | LED=000010 | change_display=0 | change_amount=0 | AN=11111011
[80365000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111011
90215000: TB - press CLEAR button (cycles=1000)
90235000: TB - select product SW=110 (id=6)
[90375000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111011
100235000: TB - press 1Y button (cycles=1000)
100265000: TB - CHECK: insufficient pay product6 no LED
[PASS] LED state: 000000
[PASS] balance: 10 (1.0 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

购买商品6, 付1元: 钱不够, 无法成功购买

```

=== TEST6: reset behavior ===
[100275000] state=0 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111110
[PASS] reset cleared state

```

检查 reset 功能

```

=== TEST7: exact payment product3(1.5Y) pay 1.5Y ===
[100285000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
110285000: TB - press CLEAR button (cycles=1000)
110305000: TB - select product SW=011 (id=3)
[110445000] state=1 | balance=10 (1.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
120305000: TB - press 1Y button (cycles=1000)
[120455000] state=1 | balance=15 (1.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[120465000] state=2 | balance=15 (1.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[120475000] state=1 | balance=15 (1.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[120985000] state=1 | balance=15 (1.5 Y) | LED=000100 | change_display=0 | change_amount=0 | AN=11111101
[121495000] state=1 | balance=15 (1.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[121995000] state=1 | balance=15 (1.5 Y) | LED=000100 | change_display=0 | change_amount=0 | AN=11111101
[122505000] state=1 | balance=15 (1.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[123005000] state=1 | balance=15 (1.5 Y) | LED=000100 | change_display=0 | change_amount=0 | AN=11111101
130315000: TB - press 0.5Y button (cycles=1000)
130345000: TB - CHECK: TEST7 exact pay product3
[PASS] LED state: 000100
[PASS] balance: 15 (1.5 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

成功购买商品3

```

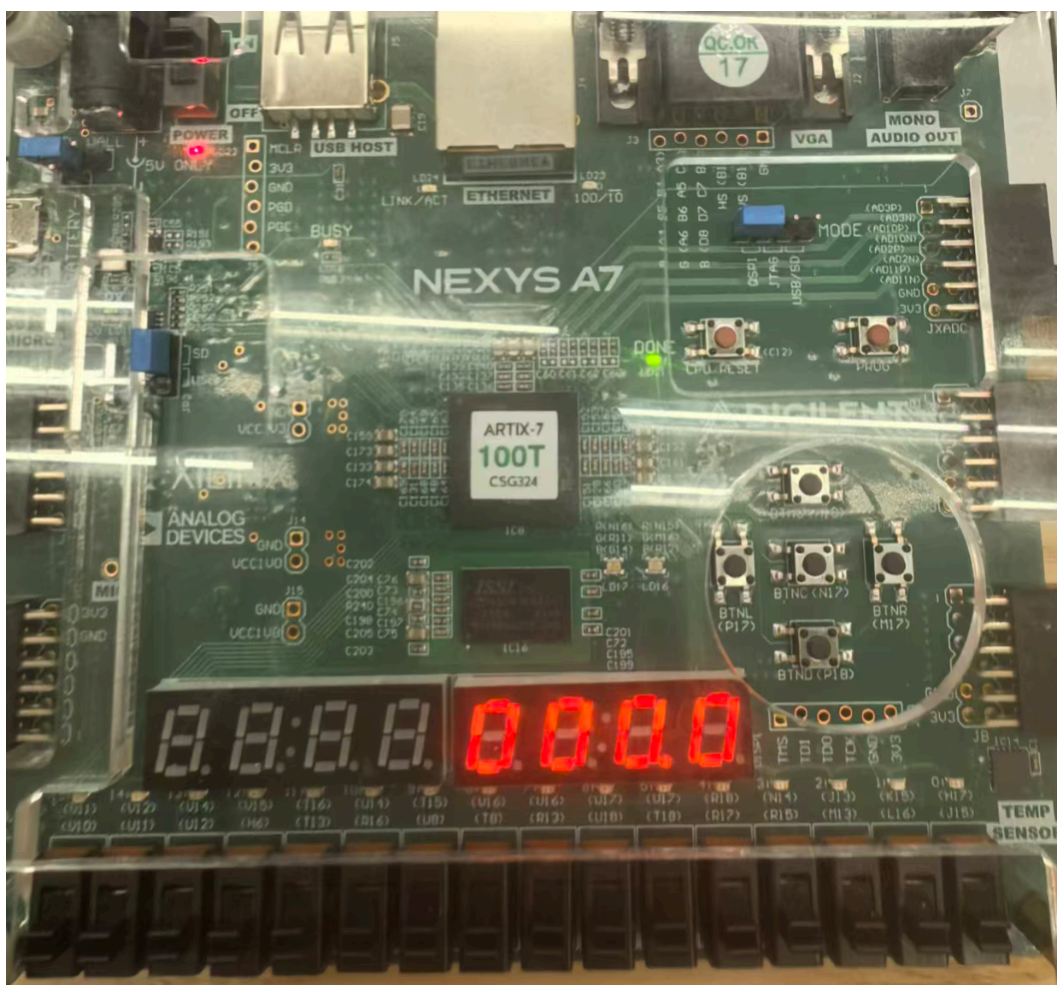
=== TEST8: exact payment product5(6.5Y) pay 6.5Y ===
[130485000] state=1 | balance=0 (0.0 Y) | LED=000100 | change_display=0 | change_amount=0 | AN=11111101
[130495000] state=1 | balance=0 (0.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
140345000: TB - press CLEAR button (cycles=1000)
140365000: TB - select product SW=101 (id=5)
[140505000] state=1 | balance=50 (5.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
150365000: TB - press 5Y button (cycles=1000)
[150515000] state=1 | balance=60 (6.0 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
160375000: TB - press 1Y button (cycles=1000)
[160525000] state=1 | balance=65 (6.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[160535000] state=2 | balance=65 (6.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[160545000] state=1 | balance=65 (6.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[161055000] state=1 | balance=65 (6.5 Y) | LED=010000 | change_display=0 | change_amount=0 | AN=11111101
[161565000] state=1 | balance=65 (6.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[162065000] state=1 | balance=65 (6.5 Y) | LED=010000 | change_display=0 | change_amount=0 | AN=11111101
[162575000] state=1 | balance=65 (6.5 Y) | LED=000000 | change_display=0 | change_amount=0 | AN=11111101
[163075000] state=1 | balance=65 (6.5 Y) | LED=010000 | change_display=0 | change_amount=0 | AN=11111011
170385000: TB - press 0.5Y button (cycles=1000)
170415000: TB - CHECK:          TEST8 exact pay product5
[PASS] LED state: 010000
[PASS] balance: 65 (6.5 Y)
internal: change_display=0 change_amount=0 change_blink_cnt=0 change_blink_timer=0

```

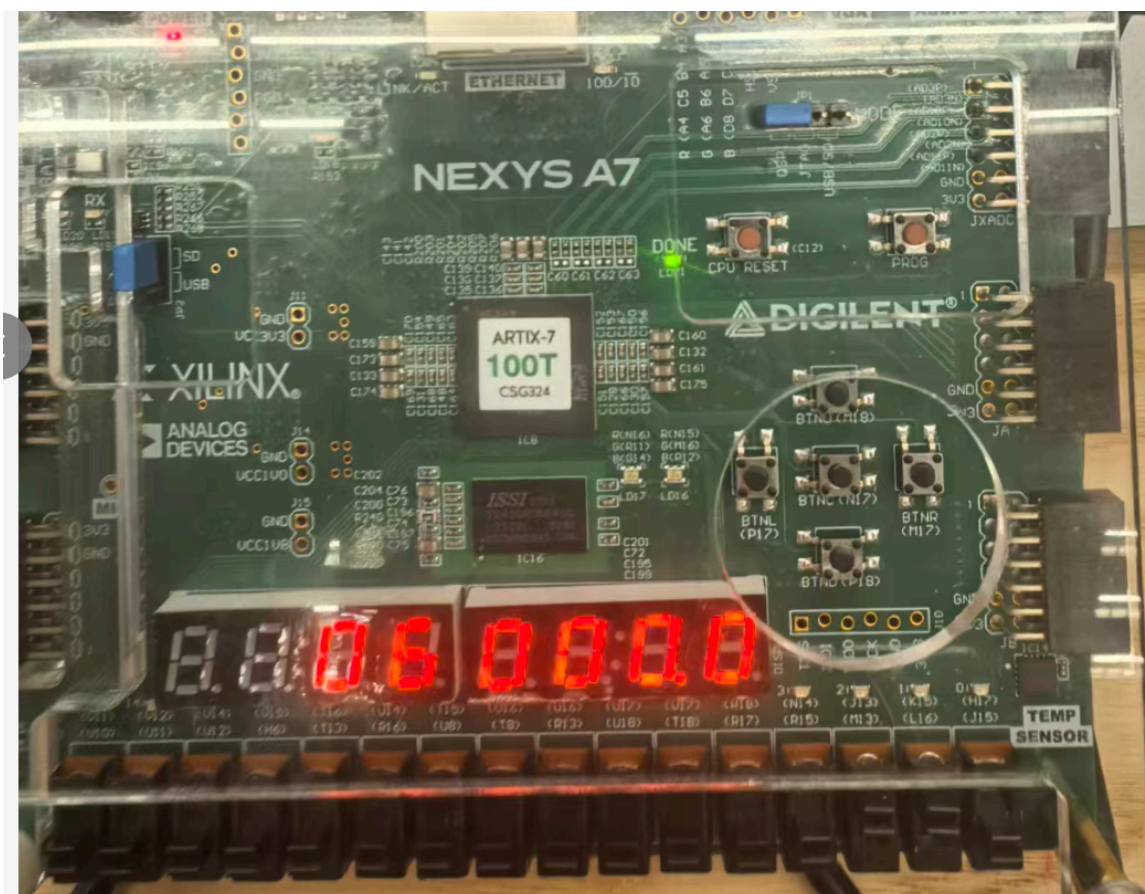
成功购买商品5

实验板验证

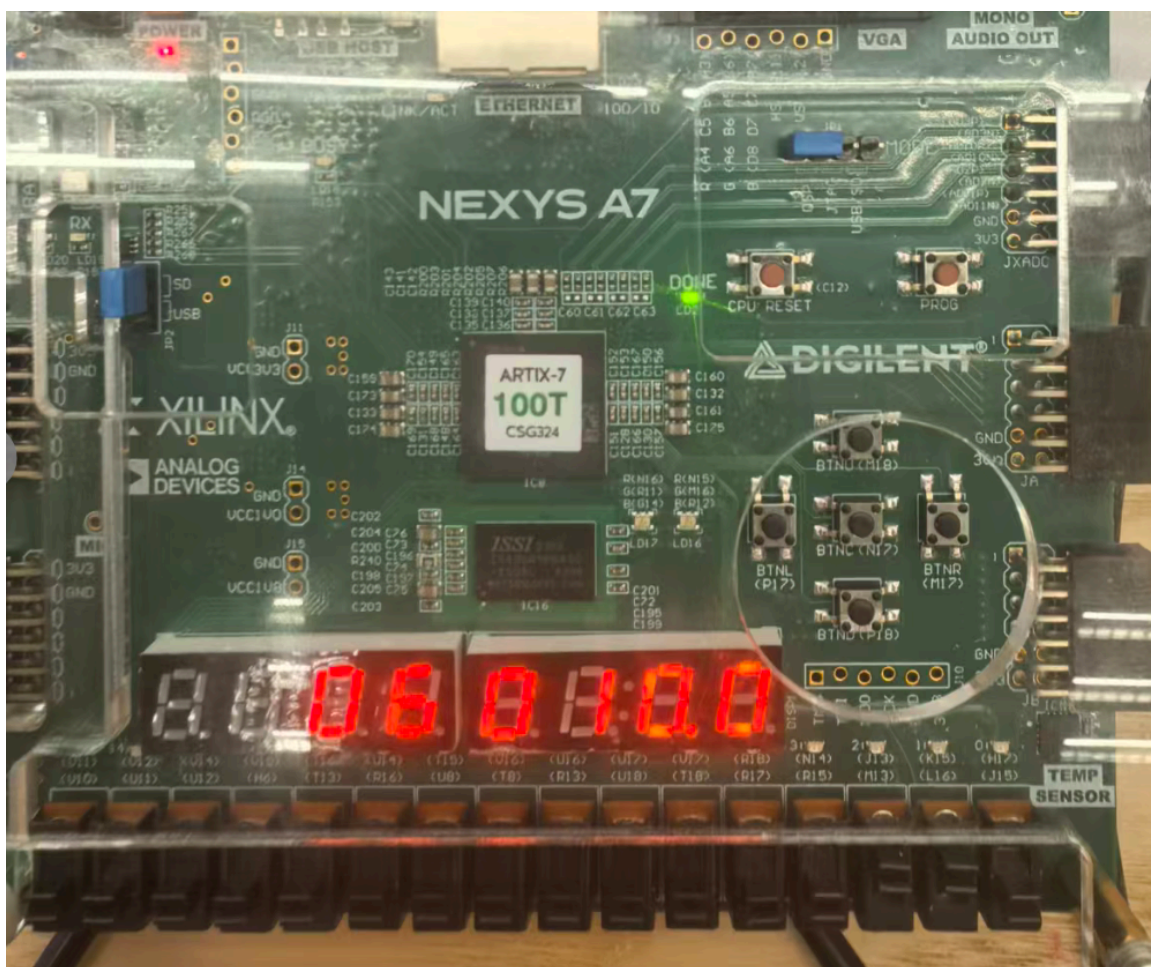
下载到实验板进行验证，可正常实现功能：



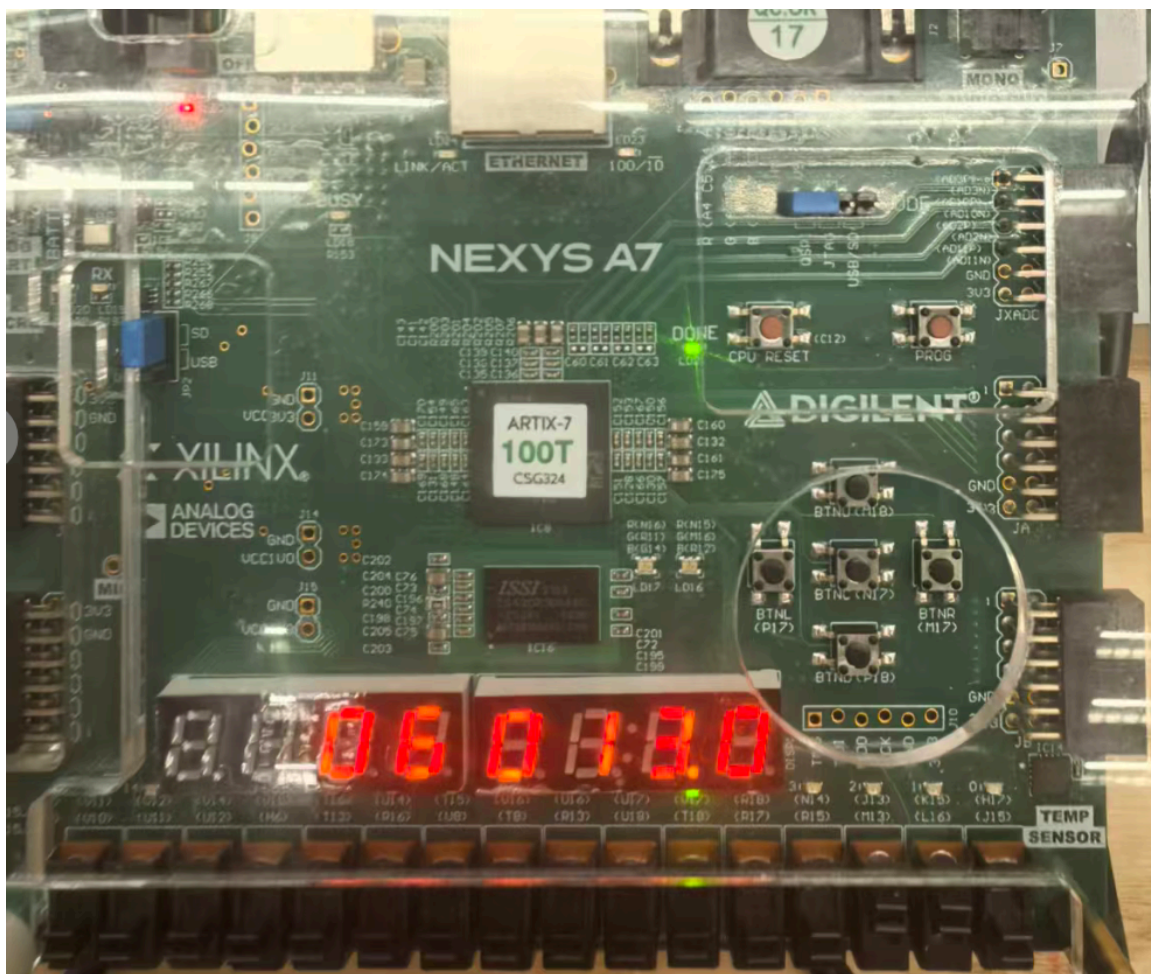
初始时：未拨动开关，左侧商品编号展示部分数码管不亮，右侧货币总和显示部分显示为000.0



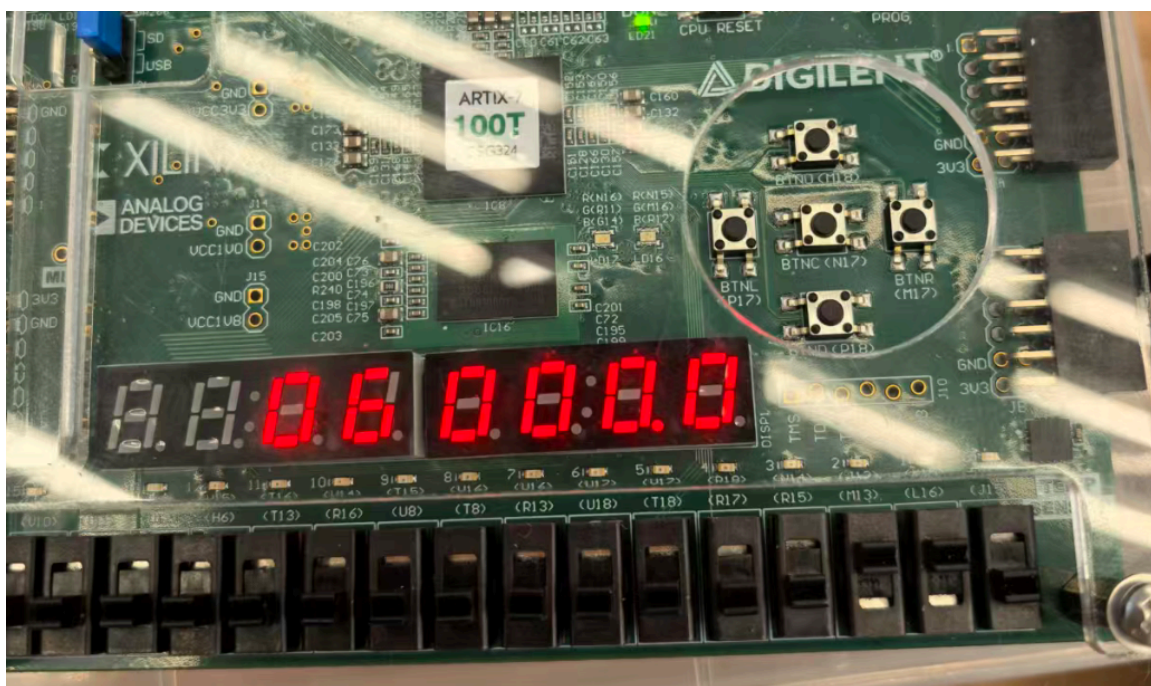
拨动开关为110，未按按钮，即选择编号为6的商品，此时左侧数码管显示06



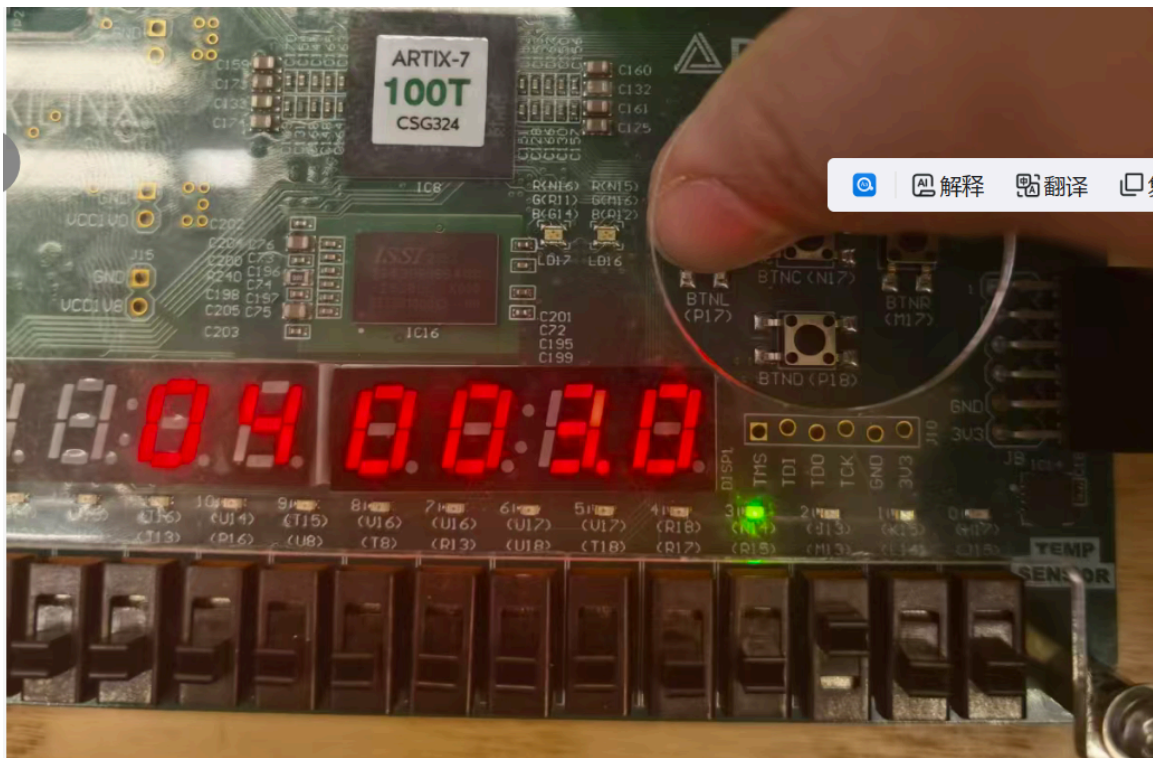
按左按钮（5元）两次，右侧数码管正确显示010.0，同时因为货币总和不等于商品价格，没有led灯亮起



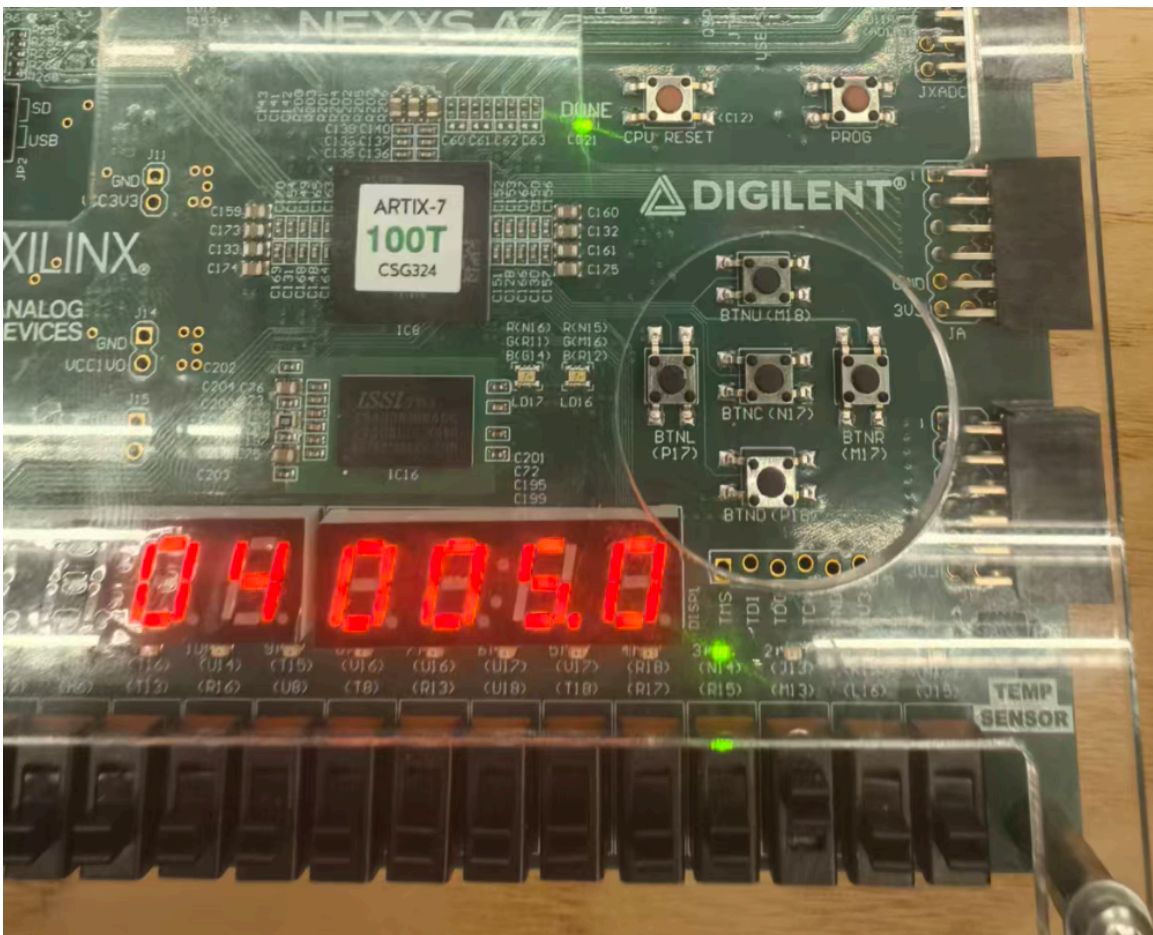
按左按钮（5元）两次，上面的按钮（1元）一次，右按钮（2元）一次，右侧数码管正确显示013.0，同时因为货币总和等于商品价格，第六个led灯正确亮起（因编号从0开始，即led[5]）



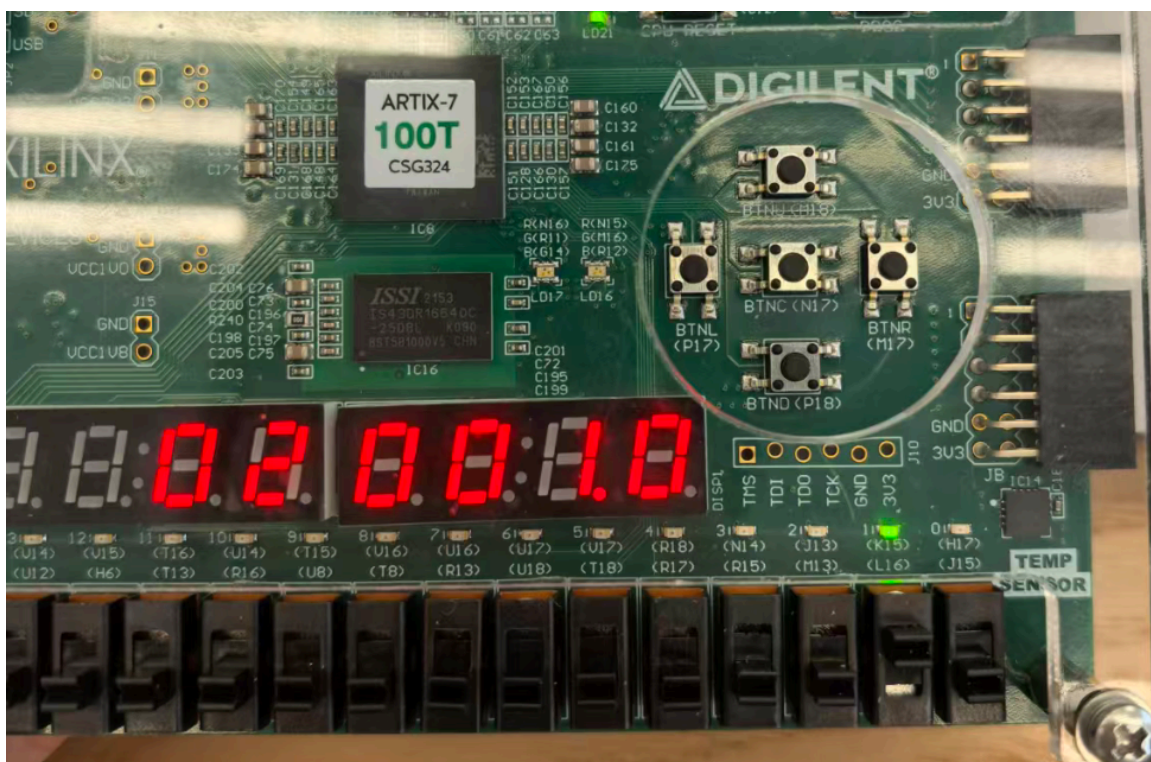
按中间按钮reset, 货币总和清零



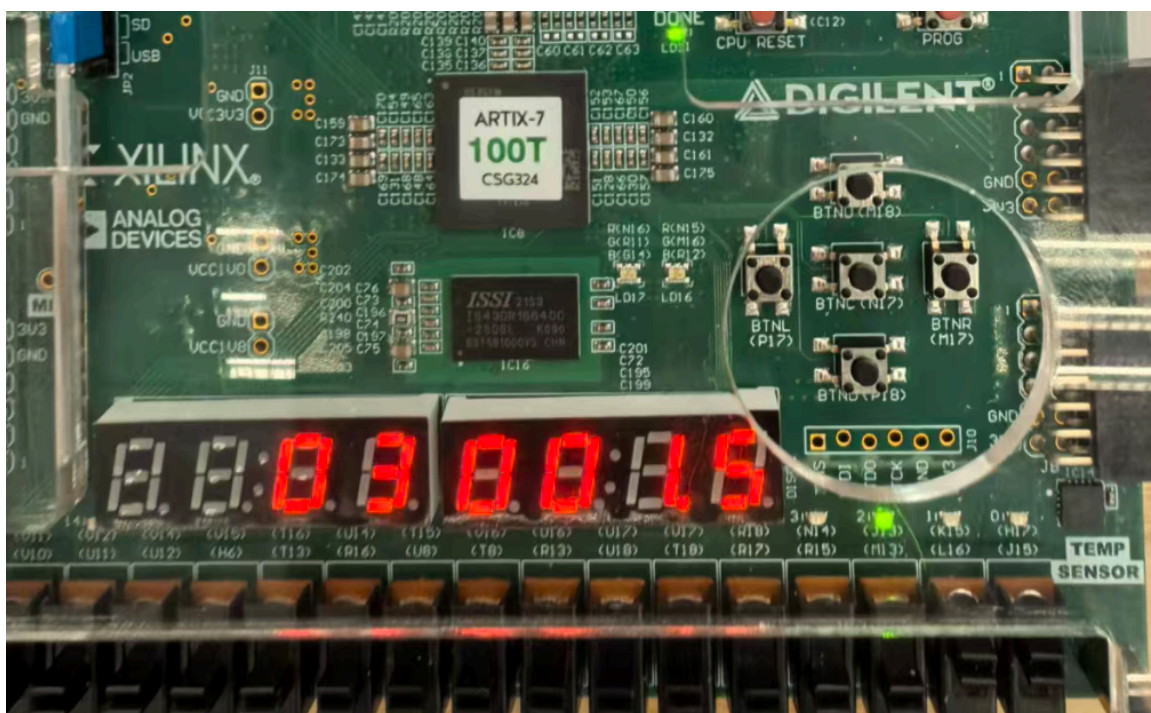
选择商品4（2元），此时按一次左按钮（5元），此时右侧数码管闪烁展示找零金额3元，led正确亮起



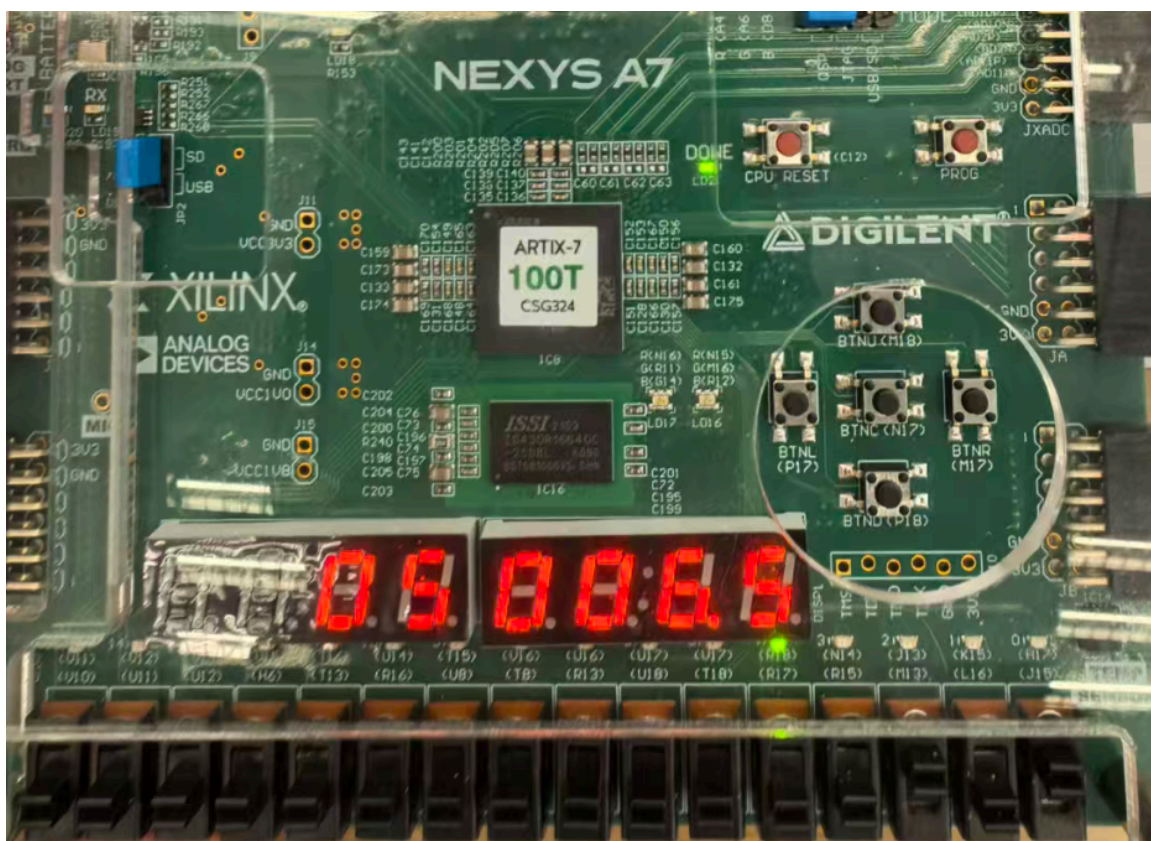
闪烁显示找零金额后，右侧数码管重新显示用户付的货币总和（5元）



成功购买商品2 (1元)



成功购买商品3 (1.5元)



成功购买商品5 (6.5元)

总结反思

遇到问题与解决分析

1. XDC 约束与端口映射问题

- 问题：约束文件存在语法/位置错误（多余或错误的括号、IO 标号不一致），导致部分 FPGA 引脚未正确约束，仿真/上板时出现 DRC 报警（UCIO-1 / NSTD-1）。
- 原因分析：约束与顶层端口名/方向不完全一致；部分位序（AN/CA..CG）理解与实际情况不同。
- 解决：修正 XDC 的 LOC 与 IOSTANDARD，核对顶层端口名与 Vivado I/O 约束逐条匹配；确认数码管位序与段极性后更新约束。

2. 数码管段向量与位选极性错误

- 问题：最初实验中 A2G 顺序/极性与板上实际（{g,f,e,d,cb,a}、共阳/共阴）不一致，导致显示错位或恒亮。
- 原因分析：文档与实际硬件位序理解反向；测试未先确认数码管类型。

- 解决：统一使用PPT上所讲解的 A2G 顺序，修正 encode_digit 函数输出与 AN 极性；在 testbench 中也把期待的 AN/segment 行为作为监控项。
- 验证：上板显示数码管正确显示“元.角”和商品编号；仿真波形段码与期望一致。

3. LED 判定逻辑 ($\geq$ vs $==$)

- 问题：早期逻辑使用“ $==$ current_price”仅接受精确支付，会导致超额支付（常见场景）无法触发购买。
- 解决：把判定条件改为“ $balance \geq current_price$ ”，满足允许超额购买的业务需求，并在 purchase 分支计算找零。
- 验证：仿真与板测均确认超额支付会触发 success_flags，并计算 change_amount。

4. debounce / edge_detect 行为与复位

- 问题：edge_detect 模块早版未对 rstn 做复位清零，去抖计数器/同步寄存器在上电可能残留垃圾，导致误触发。
- 解决：为 debounce 和 edge_detect 的 always 添加 negedge rstn 清零分支，确保上电/复位后内部状态明确；在 testbench 中引入 MIN_PRESS_CYCLES 与 C_press 确保按键脉冲足够宽以通过去抖。
- 验证：仿真注入抖动序列（短脉冲）验证去抖输出稳定，并确认 edge_detect 输出为单周期脉冲；上板验证快速按键不会导致多次触发。

5. testbench 与仿真稳定性改进（若干改动）

- 问题：最初 testbench 直接用 ns->cycles、SIM_ACCEL 混用，按键脉冲在高加速下过短，导致 DUT 未能检测到按键；另外直接 \$monitor 输出过多噪声。
- 解决与思路：
 - 引入 SIM_ACCEL 与 C_scaled、C_press，使按键最小保持周期可控 (MIN_PRESS_CYCLES)；
 - 将直接的 \$monitor 替换为事件驱动、限频的监控逻辑，减少日志噪声；
 - 把依赖于瞬态的 wait_change_display 的断言改为“延时后检查最终状态”或改进为带 re-arm 的边沿检测等待，避免错判；
 - 在部分测试用例中将 CLEAR 用复位或显式等待 balance 清零作兜底，保证每个测试有已知起始状态；

6. 显示与数字化逻辑的小错误修复

- 问题：未声明的 `digits` 数组导致综合报错；`get_digit` 中注释/边界处理需明确。
- 解决：补充 `reg [3:0] digits [0:5]`；修正 `get_digit` 注释并保证边界安全。
- 验证：综合通过并在仿真中显示数码管内容正确。

体会与建议

实验体会

- 仿真优先但不能完全代替上板验证：在功能开发阶段应先在 `testbench` 完成快速回归（使用 `SIM_ACCEL`），但最终时序行为、IO 极性与实际按键特性必须在板上验证并调参。
- 去抖与按键宽度是经常被忽视的痛点：在高加速仿真中按键脉冲会缩短到检测不到，必须在 TB 中强制最小按键保持周期（`MIN_PRESS_CYCLES`），并在板测时根据实际按键硬件调整 `DEBOUNCE_CYCLES`。
- FSM 与优先级处理要显式：把清零、复位等高优先级操作放在状态机处理逻辑的前端，能有效避免竞态与一致性问题。
- 文档与代码要同步：硬件（数码管共阳/共阴、AN 位序）与代码实现的任何偏差都会导致长时间调试浪费；每次修改约束或端口后立即做 DRC/波形确认。

具体调试建议

- 去抖参数建议流程：仿真先用较小 `N`（例如 500k）验证功能，再在板上用示波器/波形工具测量按键真实抖动窗口，最终设置 5~20 ms 范围内的合理值。
- 按键脉冲策略：上板测试时尽量模拟真实用户操作（短按 50~200 ms），不要依赖极短脉冲；TB 中用 `C_press` 强制最小宽度并把 `C_scaled` 与 `SIM_ACCEL` 明确记录。
- 数码管确认步骤：逐位点亮测试（点亮 AN 单位）并采样段信号，确认 A2G 顺序与极性再写 `encode` 表；避免“先写代码后调试”的盲目修改。